프로젝트 준비 — 개발 환경 셋업

클라우드캡스톤디자인2 · 7주차

7주차

오늘의 목표 — 바이브 코딩을 시작할 수 있는 상태 만들기

- 오늘은 실제 기능 구현(바이브 코딩)을 하지 않는다 — 구현에 필요한 환경을 전부 연결해두는 날
- 깃허브(GitHub)·버셀(Vercel)·슈퍼베이스(Supabase) 3개 서비스를 서로 연결
- 우리 팀 설계 산출물을 요약한 설정 파일을 작성해 AI 코딩 도구에게 프로젝트 맥락을 전달
- UI, [illegible]

1교시

AI 도구로 인프라 연결

Claude Code/Codex 설치 후 저장소·배포를 프롬프트로 생성

2교시

데이터베이스 연결

Supabase 연결, 설정 파일 작성 시작

3교시

설정 완성 + 화면 생성

CLAUDE.md/AGENTS.md 마무리, 화면 생성 시도

4교시

기록 환경 연결

옵시디언으로 작업 맥락 이어가기 설정

1교시 · 8분

오늘의 접근 — 환경 설정도 바이브 코딩으로

- 바이브 코딩(vibe coding): 코드를 한 줄씩 직접 타이핑하는 대신, 원하는 결과를 자연어로 설명하면 AI가 대신 처리하게 하는 방식
- 오늘은 코드뿐 아니라 저장소 만들기·배포 연결·데이터베이스 만들기 같은 "환경 설정"까지 전부 AI 코딩 도구에게 자연어로 요청한다
- 사람이 직접 하는 건 각 사이트에 로그인하는 것뿐 — 그 다음은 AI 코딩 도구가 대신 실행

왜 처음부터 AI에게 맡기는가

- 사이트 화면을 눌러가며 설정하는 방식은 사이트 UI가 바뀔 때마다 수업 내용도 따라 바뀌어야 한다
- AI 코딩 도구에게 요청하는 방식은 "무엇을 원하는지" 말로 정확히 표현하는 능력이 핵심이다
- 이 능력이 바로 바이브 코딩 전체에서 계속 쓰이는 핵심 역량 — 오늘 설정 과정이 그 첫 연습이 된다

1교시 · 7분

Claude Code·Codex란 — 터미널에서 일하는 AI

- 터미널(명령줄 인터페이스, CLI)에서 실행하는 AI 코딩 에이전트
- 자연어로 요청하면 실제 프로젝트 폴더의 파일을 읽고, 수정하고, 명령어 실행까지 함
- 웹 채팅창의 AI와 다른 점: 대화만 하는 게 아니라 내 컴퓨터 안의 파일과 프로그램을 직접 건드림
- GitHub·Vercel·Supabase 각각의 명령줄 도구(CLI)를 이용해, 오늘 할 저장소 생성·배포·데이터베이스 연결까지 대신 실행할 수 있음

1교시 · 5분

실습 — GitHub·Vercel·Supabase 로그인 확인

- 3개 사이트(GitHub·Vercel·Supabase) 전부 로그인되는지 확인 — 가입은 이미 완료된 상태
- 이메일 인증이 끝나지 않았거나 비밀번호를 잊은 경우가 가장 흔함 — 지금 바로 해결
- 오늘 수업에서 사람이 직접 하는 건 사실상 이 로그인 확인이 전부

1교시 · 5분

실습 — Claude Code 또는 Codex 설치

- 나눠드린 설치 가이드를 보며 Claude Code 또는 Codex 중 하나를 설치
- 설치가 끝나면 버전 확인 명령으로 정상 설치됐는지 확인

1교시 · 5분

실습 — 프로젝트 폴더 만들고 도구 실행하기

- 터미널에서 새 폴더를 하나 생성 — 이 폴더가 오늘부터 우리 프로젝트의 집이 됨
- 그 폴더로 이동한 뒤 Claude Code 또는 Codex를 실행
- 이후 모든 요청은 반드시 이 폴더 안에서 실행한 상태로 진행 — 다른 폴더에서 실행하면 엉뚱한 곳에 작업이 생길 수 있음

1교시 · 15분

실습 — 프롬프트로 GitHub 저장소 만들기

이렇게 요청해 보세요

“GitHub 저장소를 만들어줘. 이름은 [프로젝트명], 공개 범위는 Private으로 하고, 팀원 [아이디들]을 협업자로 초대해 줘.”

- [프로젝트명]·[아이디들] 자리는 우리 조의 실제 정보로 바꿔서 요청
- AI 코딩 도구가 GitHub의 명령줄 도구로 실제 저장소를 생성함 — 처음 한 번은 로그인 확인 창이 뜰 수 있으니 그때만 직접 확인
- 완료되면 AI가 알려주는 저장소 주소로 접속해 실제로 만들어졌는지 확인

1교시 · 15분

실습 — 프롬프트로 Vercel 연결·배포

이렇게 요청해 보세요

“방금 만든 GitHub 저장소를 Vercel에 연결해서 배포해줘.”

- AI 코딩 도구가 Vercel의 명령줄 도구로 프로젝트를 만들고 배포까지 진행 — 이번에도 처음 한 번은 로그인 확인이 필요할 수 있음
- 아직 코드 내용이 없어 빈 화면이 떠도 정상 — 주소가 발급되는지만 확인

2교시 · 15분

실습 — 프롬프트로 Supabase 연결

이렇게 요청해 보세요

“Supabase 프로젝트를 만들고 Vercel에 연결해줘. 프로젝트 이름은 [프로젝트명], 데이터베이스 비밀번호는 [비밀번호], 지역은 가까운 곳으로 해줘.”

- 비밀번호는 AI가 알려주는 값을 그대로 쓰거나 직접 정해도 됨 — 어느 쪽이든 별도로 저장해 둘 것
- AI 코딩 도구가 Supabase 프로젝트 생성부터 연결 정보(API 주소·API 키)를 Vercel에 등록하는 것까지 한 번에 진행 가능
- 오늘 목표는 "연결"까지 — 데이터베이스에 표(테이블)를 만들고 데이터를 주고받는 것은 다음 주에 진행

2교시 · 10분

연결 확인

- AI 코딩 도구에게 "지금까지 연결한 상태를 요약해서 보여줘"라고 요청
- GitHub 저장소 주소, Vercel 배포 주소, Supabase 프로젝트가 전부 연결됐는지 확인
- 빠진 항목이 있으면 그 항목만 다시 요청해서 보완

2교시 · 10분

설정 파일이란 — CLAUDE.md / AGENTS.md

- AI 코딩 도구는 프로젝트 폴더 안의 특정 파일을 세션 시작 시 자동으로 읽는다

Claude Code → CLAUDE.md

- 프로젝트 루트 폴더에 두는 텍스트 파일(마크다운, Markdown)
- 프로젝트 목표, 지켜야 할 규칙, 현재 진행 상황을 적어둠
- Claude Code를 실행하면 이 파일을 자동으로 읽고 시작함

Codex → AGENTS.md

- 같은 역할을 하는 Codex용 설정 파일
- 내용 작성 방식은 CLAUDE.md와 동일 — 도구만 다를 뿐
- 두 파일을 같은 내용으로 맞춰두면 어느 도구를 써도 동일하게 동작

2교시 · 25분

실습 — CLAUDE.md/AGENTS.md 작성 시작

- 아까 만든 프로젝트 폴더 안에 CLAUDE.md·AGENTS.md 두 파일을 생성
- 배포된 작성 양식의 4개 항목(프로젝트 개요·핵심 기능·데이터 구조·지켜야 할 규칙)을 채우기 시작
- 각 항목은 이미 완성한 설계 문서에서 핵심만 요약해 옮겨 적으면 됨 — 새로 만드는 내용이 아님

3교시 · 20분

CLAUDE.md/AGENTS.md 작성 마무리

- 4개 항목(프로젝트 개요·핵심 기능·데이터 구조·지켜야 할 규칙)을 전부 채움
- 두 파일의 내용이 서로 다르지 않은지 마지막으로 대조
- 완성한 파일은 AI 코딩 도구에게 "GitHub에 커밋하고 푸시해줘"라고 요청해 저장소에 올리고, 팀원 전체가 같은 내용을 공유

3교시 · 15분

실습 — 설정 파일 인식 확인

1

프로젝트 폴더에서 도구 다시 실행

설치는 1교시에 끝났으므로, 프로젝트 폴더에서 바로 실행

2

인식 확인 질문

"이 프로젝트가 뭘 하는 프로젝트인지 설명해줘"라고 물어보기

3

결과 비교

CLAUDE.md/AGENTS.md 내용과 AI의 설명이 일치하는지 확인

3교시 · 15분

실습 — 화면(UI) 프로토타입 생성 시도

- 가장 핵심이 되는 화면 1개만 선택 — 로그인, 목록, 메인 화면 등
- "기능은 연결하지 말고, 화면 레이아웃만 만들어줘"라고 명확히 요청
- 데이터베이스 연결·버튼 동작 등 실제 기능은 이번에 요청하지 않음
- 화면이 완성되지 않아도 괜찮음 — 오늘의 목적은 도구가 실제로 동작하는지 확인하는 것

3교시 · 10분

조별 진행상황 확인

- 각 조가 지금까지 끝낸 항목을 서로 확인
- ① GitHub 저장소(프롬프트로 생성) ② Vercel 배포 ③ Supabase 연결 ④ CLAUDE.md/AGENTS.md ⑤ 설정 파일 인식 확인 ⑥ 화면 생성 시도
- 안 끝난 항목이 있으면 4교시 중간에도 이어서 진행 가능 — 오늘 안에 전부 끝내는 것이 목표가 아니라, 전체 흐름을 한 번 경험하는 것이 목표

4교시 · 15분

옵시디언(Obsidian)이란 — 작업 맥락을 이어가는 기록

- 마크다운 기반 메모 앱 — 오늘 만든 CLAUDE.md/AGENTS.md와 같은 형식의 파일을 모아두는 공간

왜 필요한가

- 바이브 코딩은 여러 날, 여러 세션에 걸쳐 진행됨
- 매번 "지난번에 어디까지 했더라"를 다시 설명해야 하면 비효율적
- 작업 내용을 기록해두는 공간(vault, 저장소)이 있으면 그 문제가 해결됨

체크인 · 체크아웃

- 체크아웃: 오늘 한 작업·결정 사항을 기록에 저장
- 체크인: 다음 세션 시작할 때 그 기록을 불러와 맥락을 복원
- 이 옵시디언 자료 자체가 그 방식으로 운영되고 있는 실제 사례

4교시 · 35분

실습 — 옵시디언 vault 설정 + 프로젝트 연동

1

옵시디언 설치

안내에 따라 옵시디언 앱 설치

2

vault 생성

프로젝트 폴더 바로 옆(또는 안)에 작업 기록용 폴더 생성

3

기본 문서 작성

오늘 한 작업 요약을 첫 기록으로 남기기(배포된 양식 참고)

4

연동 구조 확인

프로젝트 폴더와 vault 폴더 경로가 서로 연결되는지 확인

4교시 · 10분

오늘 전체 정리

- 코드 저장소(GitHub)·배포 서비스(Vercel)·데이터베이스(Supabase)를 전부 연결
- AI 코딩 도구가 프로젝트를 이해하도록 CLAUDE.md/AGENTS.md를 작성
- 도구를 설치해 프로젝트를 인식시키고, 화면 생성을 한 번 시도
- 작업 맥락을 이어갈 기록 공간(옵시디언 vault)까지 준비 완료

지금 손에 남은 것

— 코드가 올라갈 저장소, 화면이 뜰 배포 주소, 데이터를 담을 데이터베이스가 전부 연결된 상태

— AI 코딩 도구가 프로젝트를 이해하는 설정 파일과, 작업을 기록할 공간